

Isabella Mastroeni

LINGUAGGI

Cap. 2 DESCRIVERE I LINGUAGGI

```
5
def(ast):
    e = getNodeName()
    symbol.sym_name.get(int(ast[0]), ast[0])
    %s [label="%s" % (nodename, la
instance(ast[1], str):
    ast[1].strip():
    print '%s' % ast[1]
else:
    print ""

print ""
children = []
for n, child in enumerate(ast[1:]):
    children.append(dotwrite(child,
    %s -> { % nodename,
print
for name in children:
    print '%s' % name,
```

COME SI DESCRIVONO LINGUAGGI?

- Cosa va descritto?
- Con quali strumenti?

- Un linguaggio di programmazione è un formalismo artificiale mediante il quale esprimono algoritmi. Anche se artificiale, è sempre un linguaggio e quindi deve essere descritto usando noti strumenti della linguistica.
- Un linguaggio di programmazione non è altro che come un linguaggio naturale, ma semplificato e non ambiguo. La teoria dei linguaggi di programmazione quindi deve parlare di:

- Grammatica (**sintassi**)

Regole di formazione:
quando una frase è corretta
= relazioni tra segni

- **Semantica**

Attribuzione di significato:
cosa significa una frase corretta
relazioni tra segni e significato

- **Pragmatica**

In qual modo frasi corrette e sensate
sono usate
relazioni tra segni, significato e utente

- **Implementazione**

eseguire una frase corretta,
rispettandone la semantica

Semantica, pragmatica e implementazione sono elementi da studiare separatamente ma legati dalla sintassi in funzione della quale tutti sono definiti.

DESCRIVERE I LINGUAGGI DI PROGRAMMAZIONE

La descrizione di un linguaggio avviene su tre dimensioni (Morris, 1939):

- **Sintassi:** è l'insieme di regole che permettono di costruire frasi corrette. Ogni frase è costituita da componenti che rappresentano le categorie sintattiche (nel linguaggio naturale soggetto, verbo, oggetto...). Dopo di che, individuato l'alfabeto, a livello lessicale si individuano le sequenze di simboli legali, ovvero quelle che costituiscono le parole del linguaggio. La fase che determina la corretta struttura delle frasi, ovvero che verifica se una frase rispetta le regole della grammatica, è detta *parsing*.
- **Semantica:** Nei linguaggi artificiali l'attribuzione del significato è una fase semplice ed è la definizione di una relazione tra segni (frasi legali) e significati (entità autonome che esistono indipendentemente dai segni che usiamo per descriverle). Ad esempio la semantica di

un programma può essere la funzione matematica calcolata dal programma. Solitamente viene specificata descrivendo gli effetti della sintassi su una rappresentazione astratta della macchina, chiamata *stato*.

- **Pragmatica:** Frasi con lo stesso significato possono essere usate in modo diverso in modo dipendente dal contesto linguistico.
- **Implementazione:** Questi aspetti riguardano la tecnica di implementazione usata, i vincoli architettura o della macchina (ad es. MAXINT, gestione della memoria, architettura basata su stack o su registri), l'interfaccia del SO (gestione dell'input/output, struttura dei file, ecc), gestione degli errori (natura e metodi di gestione). Quindi qui stiamo considerando tutti quegli aspetti che hanno effetto sul funzionamento del linguaggio ma che dipendono dalla macchina su cui il linguaggio viene eseguito.

SINTASSI

Le regole sintattiche del linguaggio specificano quali stringhe di caratteri sono legali nel linguaggio. Nel dettaglio la terminologia nella linguistica è la seguente:

- Una *parola* è una stringa di caratteri su un alfabeto
- Una *frase* è una sequenza (ben formata) di parole
- Un *linguaggio* è un insieme di frasi

Nell'ambito dei linguaggi di programmazione gli stessi concetti prendono nomi differenti, in particolare:

- Le parole diventano *lessemi*. Un lessema è una parola con significato specifico, nella grammatica corrisponde ad un terminale. È l'unità minima sintattica, ovvero quella a più basso livello di un linguaggio di programmazione (es. `*`, `sum`, `begin`...). La loro specifica sintattica è separata dalla definizione del linguaggio, e come vedremo, solitamente descritta come stringhe di

linguaggi regolari. Ad esempio, in `index=2`, abbiamo che `index`, `=` e `2` sono lessemi.

- Le frasi diventano *token*. I token corrispondono agli elementi delle categorie sintattiche del linguaggio di programmazione, e nella grammatica corrispondono alle sequenze generate dai simboli non terminali (ad es. identificatore, comando,...). Ad esempio, `index=2` è un token (assegnamento/comando).
- Il *programma* è una sequenza/composizione sequenziale (nella grammatica) di frasi ben formate.
- Il linguaggio dei programmi, e quindi gli strumenti formali che lo definiscono, costituiscono quello che chiamiamo *linguaggio di programmazione*.

Questi concetti chiaramente non servono direttamente a chi usa il linguaggio, ma servono a chi lo deve implementare.

DESCRIVERE LA SINTASSI

Abbiamo bisogno di strumenti formali per descrivere parole e frasi legali di un linguaggio, che permettano in modo automatico di verificare l'appartenenza o meno al linguaggio. In particolare, le parole sono stringhe di simboli alfabetici, mentre le frasi sono stringhe di parole.

Nei linguaggi di programmazione, il linguaggio dei lessemi è in generale sempre un linguaggio **regolare**, quindi **riconosciuto** da un automa a stati finiti.

Riconoscitore:

Uno strumento di riconoscimento legge in input stringhe sull'alfabeto del linguaggio e decide se la stringa appartiene o meno al linguaggio. Esempio: analisi sintattica (che riconosce lessemi) di un compilatore (AST).

Supponiamo di avere un linguaggio L su un alfabeto Σ , per costruire un metodo di riconoscimento dovremmo avere un meccanismo R in grado di leggere le stringhe di

caratteri. R quindi deve essere in grado di dire se una stringa è in L oppure no.

Invece, il linguaggio dei token, e quindi dei programmi, è in generale un linguaggio **context-free (CF)**, quindi **generato** da una grammatica CF. Possiamo pensarlo come uno strumento con un bottone che genera una stringa (in modo non deterministico) quando si preme il bottone.

Generatore:

Uno strumento che genera stringhe di un linguaggio.

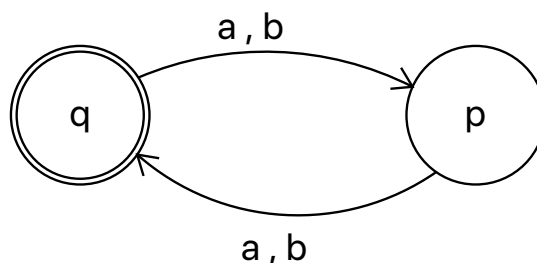
Si può determinare se la sintassi di una particolare frase è sintatticamente corretta confrontandola con la struttura del generatore (parser)

Possiamo comunque osservare che anche i linguaggi regolari possono essere generati (grammatiche regolari) e i linguaggi CF possono essere riconosciuti (automi a pila).

ESEMPIO

$$L = \{ \sigma \in \Sigma^* \mid |\sigma| \text{ pari} \}$$

$$\Sigma = \{a, b\}$$



$$S := \epsilon \mid aaS \mid abS \mid baS \mid bbs$$

Ma vediamo quali sono questi strumenti formali, in particolare per descrivere il linguaggio come sequenza di parole/lessemi. Come si descrive quindi la sintassi? Nel linguaggio naturale viene usato il linguaggio naturale stesso, ma il linguaggio naturale introduce ambiguità e non può essere di aiuto all'implementazione di un linguaggio di programmazione. Negli anni '50, Chomsky ha sviluppato un formalismo al fine di descrivere i linguaggi artificiali in modo formale, permettendo di limitare le ambiguità: GRAMMATICHE GENERATIVE, molto utilizzate per descrivere la sintassi dei linguaggi di programmazione.

Supponiamo di voler descrivere i numeri, possiamo prendere le cifre arabe (126) o i numerali romani (CXXVI), nel primo caso abbiamo un sistema di numerazione posizionale nel secondo no. Qui noi descriviamo lo stesso insieme di elementi astratti (numeri) ma in forme fisiche diverse. Cosa le distingue? L'alfabeto sicuramente, ma non solo, anche la grammatica che ci permette di generare i numeri, cambia quindi il modo con cui otteniamo forme composte (numeri a più cifre) a partire dalle cifre semplici. Quindi cambia sia l'alfabeto (ovvero il modo con cui rappresentiamo le cifre) che le regole (come combiniamo le cifre per ottenere i numeri). Diverso l'esempio tra

sistema binario e decimale (o esadecimale): in questo caso cambia l'alfabeto (e quindi la rappresentazione delle cifre semplici) ma non cambia l'insieme delle regole, il modo con cui combiniamo le cifre (posizionale) per ottenere i numeri. D'altra parte, i linguaggi possono avere anche diversi insiemi di caratteri, ma con lo stesso insieme di elementi astratti rappresentati dalle parole (come per lingue diverse).

Il vocabolario è la base per costituire un dizionario completo delle parole del linguaggio. La grammatica descrive i modi per combinare parole e formare frasi, ed è descritta da:

- *Alfabeto*
- *Descrizione lessicale*: Sequenze di simboli corretti che costituiscono le **parole/lessemi** del linguaggio (ovvero i simboli terminali della grammatica)
- *Descrizione delle sequenze di parole* che costituiscono frasi legali del linguaggio (programmi)

Diversi formalismi esistenti:

- Grammatiche libere dal contesto (notazione BNF)

ESEMPIO

$$L = \{\sigma \in \Sigma^* \mid \sigma \text{ palindromo}\}$$

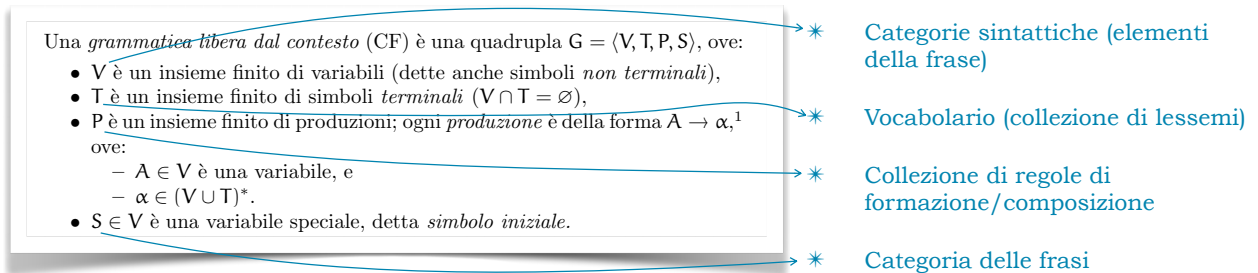
Def: • ϵ , a e b sono palindromi
 • se σ è palindromo allora lo
 sono anche $a\sigma a$ e $b\sigma b$

$$S := \epsilon \mid a \mid b \mid aSa \mid bSb$$

GRAMMATICHE CONTEXT-FREE

Abbiamo visto che le grammatiche CF permettono di descrivere la sintassi dei linguaggi di programmazione. Questo generatore di linguaggi è stato sviluppato da Noam Chomsky attorno al 1950 per descrivere la sintassi dei linguaggi naturali, e permettere di definire la classe dei linguaggi context-free (CF).

elementi che possono essere usati per comporre frasi. Le produzioni sono le regole di composizione degli elementi in una frase, formalmente sono utilizzate per rimpiazzare un non terminale con una stringa di uno o più simboli terminali e/o non terminali. Il linguaggio è l'insieme di tutte le possibili frasi, ovvero le stringhe di simboli terminali



Nella grammatica i simboli terminali costituiscono le parole/lessemi del nostro linguaggio, ovvero il vocabolario, invece i simboli non terminali sono le categorie sintattiche, quindi rappresentano i diversi tipi di

(parole), generate a partire dal simbolo iniziale S , che rappresenta la categoria delle frasi legali nel linguaggio (dei programmi nei linguaggi di programmazione).

Ma cosa significa context free? Significa **libero dal contesto**, questo è un vincolo sulle produzioni, dove a sinistra possiamo trovare un solo non terminale, e questo implica che in una frase, dovunque troviamo quel non terminale, ovvero in **qualsunque contesto** troviamo quel non terminale, lo possiamo sostituire/rimpiazzare applicando una delle produzioni per il non terminale. In altre parole, il contesto in cui un non terminale A si trova non pone nessun vincolo sulle produzioni per A che noi possiamo applicare. Nei linguaggi di programmazione, questo significa che il modo con cui possiamo sviluppare, per sostituzione, una categoria sintattica è indipendente dal contesto.

deve facilitare la definizione semantica del linguaggio (ad esempio grammatiche lineari destre). Esempio: Numeri arabi $S \rightarrow dS \mid S \rightarrow d, d \in \{0,1,2,3,4,5,6,7,8,9\}$.

Purtroppo, la proprietà CF è anche uno svantaggio per i linguaggi di programmazione, infatti tali grammatiche non riescono a catturare vincoli contestuali, che pure sono necessari. Ad esempio il poter usare una variabile solo se questa è stata precedentemente dichiarata/definita. Questo, significa che abbiamo bisogno di altri formalismi per descrivere/verificare questa tipologia di vincoli su cui ritorneremo.

Questo è un vantaggio perché dato un programma abbiamo il problema di capire quale grammatica può averlo generato (*parsing*), e per le grammatiche CF esistono strumenti automatici efficienti di parsing. Dal punto di vista della complessità del parsing, questa dipende dal numero di simboli terminali della grammatica e dal grado di non determinismo delle produzioni. Un linguaggio può essere generato da diverse grammatiche, quindi ai fini dell'efficienza del linguaggio è importante come si sceglie la grammatica. Per prima cosa deve essere semplice, il più possibile: CF, basso non determinismo,

$$G = \{\{E\}, \{\text{or, and, not, (,), 0, 1}\}, P, E\}$$

$$E \rightarrow 0|1|(E \text{ or } E)|(E \text{ and } E)|(\text{not } E).$$

$$E \rightarrow 0$$

$$E \rightarrow 1$$

$$E \rightarrow (E \text{ or } E)$$

$$E \rightarrow (E \text{ and } E)$$

$$E \rightarrow (\text{not } E)$$

NOTAZIONE BNF

La BNF è un metalinguaggio (linguaggio usato per descrivere altri linguaggi) per descrivere linguaggi di programmazione. È stata introdotta a partire da Algol60, prima (con FORTRAN) quello che succedeva era che chi progettava ogni implementazione di un compilatore dava significati diversi agli stessi costrutti, questo significava che programmi FORTRAN, sviluppati per macchine diverse, non erano confrontabili. Quindi spostare programmi da una macchina con un compilatore ad una con un altro diventava un grosso problema, che implicava riscrivere interamente il programma per adattarsi al nuovo compilatore o riscrivere il compilatore per adattarsi alla nuova macchina.

Questa notazione è usata per descrivere grammatiche CF, dove si usano intere parole come simboli terminali, i non terminali sono identificati racchiudendoli tra parentesi angolate $\langle \rangle$, e per le produzioni si usa il simbolo $::=$ al posto della freccia.

$$\begin{aligned}\langle A \rangle &::= \alpha \langle B \rangle \gamma \mid \alpha \\ \langle B \rangle &::= \varepsilon \mid \beta_1 \mid \beta_2\end{aligned}$$

Dal punto di vista del significato, è esattamente quello che abbiamo già descritto per le grammatiche. Quindi i *non terminali* sono astrazioni usate per rappresentare classi di strutture sintattiche, i simboli *terminali* sono lessemi e ogni regola ha una parte sinistra (LHS) contenente un non terminale, e una parte destra (RHS) che è una stringa di terminali e non terminali. La notazione BNF è quindi semplice ma sufficientemente potente per rappresentare linguaggi di programmazione: rappresenta l'ordine degli operatori, le strutture annidate ad ogni livello, le regole di precedenza e di associatività degli operatori. Per semplificare la rappresentazione si usa anche la EBNF che aggiunge la possibilità di rappresentare *opzioni*: $[]$ indica 0 o 1 occorrenza di quanto contenuto; $\{ \}$ indica 0 o più occorrenze di quanto contenuto; la virgola permette di esprimere più opzioni in or.

$$\langle A \rangle ::= \alpha [\beta_1, \beta_2] \gamma \mid \alpha \gamma \mid \alpha$$

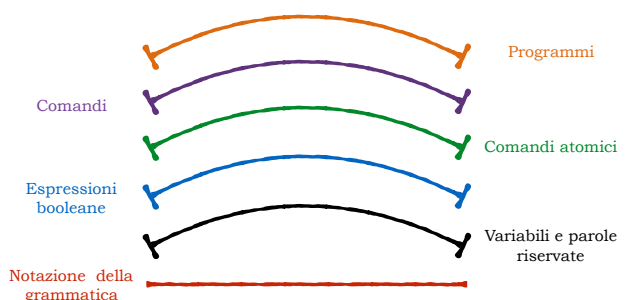
DESCRIVERE UN SEMPLICE LINGUAGGIO IMPERATIVO

Un linguaggio può essere descritto anche in modo informale, descrivendo quali caratteristiche vogliamo inserire e quale è il loro significato:

- Niente dichiarazioni
- Solo variabili ed espressioni booleane
- Assegnamento e composizione sequenziale
- Comando condizionale
- Comando iterativo (loop)

Questo può essere sufficiente per scrivere un programma corretto che un altro programmatore può leggere, ma non è adeguato per costruire un compilatore o un interprete. Per permettere l'implementazione della macchina astratta abbiamo bisogno di scendere ad un livello più formale.

Ritorna il concetto di astrazione, ed in particolare osserviamo che ogni linguaggio è descritto a più livelli di *astrazione* ognuno descritto da una grammatica.



<program>	S → C
<com>	C → A C ; C if B then C else C while B do C
<atomic com>	A → v := B
<b-expr>	B → true false v (not B) (B and B) (B or B)

Le frasi del linguaggio sono i programmi, niente dichiarazioni e niente preamboli. Un programma è qualunque comando e ogni comando è un programma. Ad ogni livello di specifica della grammatica quanto specificato alle grammatiche sottostanti è trattato da terminale ma per avere una definizione completa dobbiamo necessariamente definire esplicitamente anche questi livelli. Quindi, ad esempio, nella grammatica dei comandi (C) usiamo la categoria sintattica dei comandi atomici (A), questo significa che ogni frase generata da A è trattato come un terminale dalla grammatica per C. Analogo per le espressioni booleane (B).

Nelle espressioni abbiamo una notazione completamente "parentesizzata" per evitare ambiguità, se togliamo le

B → **v** | **not B** | **B and B** | **B or B**

parentesi introduciamo ambiguità. L'ambiguità consiste nell'esistenza di più parse-tree per la stessa espressione. Ad esempio, consideriamo l'espressione $v_1 \text{ or } v_2 \text{ and } v_3$, abbiamo ambiguità in quanto l'albero può avere come radice sia l'or che l'and, ottenendo alberi (e risultati di valutazione ad esempio con $v_1=tt, v_2=tt, v_3=ff$) diversi.

Chiaramente l'ambiguità è da evitare perché vogliamo che ogni costruito abbia un significato unico. In ogni caso spesso le grammatiche sono ambigue per semplificare la rappresentazione assumendo implicitamente le convenzioni di precedenza ereditate dalla matematica. Notare che in questo linguaggio ci sono ambiguità anche nei comandi, ad esempio nella composizione sequenziale. In questo caso poiché i comandi rappresentano funzioni, si assume la proprietà associativa delle funzioni nella composizione sequenziale di comandi.

ANALISI "SEMANTICA"

Ritorniamo sui limiti del formalismo che usiamo per descrivere i linguaggi di programmazione, ovvero l'essere CF. Abbiamo già osservato come ci siano vincoli che dipendono dal contesto:

- Il numero di parametri attuali di una chiamata di procedura deve essere uguale al numero dei parametri formali della dichiarazione;
- $I=R+3$; questa potrebbe essere sintatticamente corretta ma illegale nel contesto in cui si trova. Ad esempio se il linguaggio fosse fortemente tipato e prima non ci fosse una dichiarazione di R e di I il comando sarebbe illegale;
- I tipi della variabile e dell'espressione di un assegnamento devono essere compatibili.

Quindi stringhe corrette per una certa grammatica sono legali solo in determinati contesti. Questo è chiaramente

un vincolo sintattico, ma lo strumento sintattico scelto (grammatiche CF) non permette di descriverlo.

Possiamo percorrere diverse strade per risolvere la questione:

- Usare strumenti più potenti, per esempio grammatiche contestuali: riscrittura di un non-terminale solo in un determinato contesto
- Usare controlli *ad hoc*

Il problema delle grammatiche contestuali (CSG) è che non esistono algoritmi lineari per il riconoscimento di stringhe generate, quindi non ci sono algoritmi efficienti per fare il parser delle stringhe di una CSG. Per questo la scelta ricade su una specifica del linguaggio mediante grammatica CF (**sintassi**) e poi, come parte della semantica (da cui il nome di vincoli semantici o **semantica statica**), vengono specificati i vincoli contestuali.

SEMANTICA (DINAMICA)

La semantica è più complessa della sintassi perché ricerca esattezza e flessibilità.

- **Esattezza:** Descrizione precisa e non ambigua di cosa ci si debba aspettare da ogni costrutto sintatticamente corretto, per sapere a priori quello che succederà durante l'esecuzione.
- **Flessibilità:** Non deve anticipare scelte che possono essere demandate all'implementazione.

Non esiste un singolo formalismo universalmente accettato per dare significato ai programmi, ma esistono diverse necessità di metodologia e notazione per descrivere significati:

- I programmatori devono sapere cosa significano i comandi
- Gli sviluppatori devono sapere esattamente cosa fanno i costrutti

- Gli sviluppatori potrebbero rilevare ambiguità e inconsistenze
- Dimostrazioni di correttezza dovrebbero essere possibili
- Generatori di compilatori dovrebbero essere possibili

Quella che dobbiamo formalizzare per rispondere a tutte queste problematiche è la **semantica** del linguaggio di programmazione, a volte anche chiamata semantica dinamica per distinguerla da quella statica (vincoli semantici). *Perché serve quindi?* Dobbiamo specificare gli effetti della sintassi sulla rappresentazione astratta della macchina, chiamata **stato**. Quindi, per ogni costrutto, va descritto il *significato della sua esecuzione come trasformazione di stato*. L'astrazione della macchina nel concetto di stato permette di dare al costrutto un significato puro, indipendente dalla macchina, quindi senza restrizioni.

Semantica: Attribuisce un significato ad ogni frase sintatticamente corretta.

Significati: Entità autonome che esistono indipendentemente dai segni che usiamo per descriverle

La semantica guarda quindi al linguaggio di programmazione ad un livello ideale, indipendente dalla macchina, con l'unico vincolo che in ogni istante possono essere stati eseguiti un numero finito di passi di computazione. Chiaramente, non è facile trovare il giusto equilibrio tra esattezza e flessibilità, in modo da rimuovere ambiguità, ma anche lasciare spazio all'implementazione.

Come possiamo quindi dare significato ad un linguaggio di programmazione? Di certo non possiamo dare un significato esplicito ad ogni frase del linguaggio essendo queste infinite, quindi anche la semantica deve fornire una rappresentazione finita del significato dei programmi a partire dalla sua sintassi (rappresentazione finita di un insieme infinito). Abbiamo visto che la sintassi è descritta attraverso un vocabolario (di elementi primitivi) e un insieme di regole per combinare le parole del vocabolario (produzioni che combinano categorie sintattiche in una frase e le istanziano nelle parole del vocabolario). Anche la semantica quindi, per essere descritta in modo finitario, pur dando significato ad un insieme infinito deve basarsi sulla struttura costruttiva della sintassi. Deve dare

semantica agli elementi primitivi e deve descrivere il significato di una forma composta in funzione del significato degli elementi composti. Quindi in generale il significato di un programma è la composizione del significato dei costrutti che lo compongono. Il significato di ogni costrutto è dato in termini del significato delle categorie sintattiche e quindi delle parole che lo compongono, e così via fino ad arrivare ad usare il significato di un vocabolario finito e noto. In sintesi, quindi per dare semantica dobbiamo sempre dare significato agli elementi complessi in funzione del significato degli elementi più semplici che lo compongono e del modo con cui questi sono composti.

Esiste quindi un legame forte tra sintassi e semantica: Sintassi come metodo finitario per rappresentare un insieme infinito di programmi, la cui sola cosa analizzabile è la struttura; Semantica come metodo finitario (per questo dato seguendo la struttura della sintassi) per dare significato a tutti gli elementi dell'insieme infinito dei programmi.

INDUZIONE

Quindi la semantica va data seguendo la struttura della sintassi, e la sintassi è definita descrivendo gli elementi base e componendo questi elementi, attraverso regole, in elementi composti. Questa forma di definizione ha una connotazione ben precisa nella matematica, ed è chiamata **induzione**. Si usa sempre un ordine ben fondato perché questo garantisce di trovare sempre un minimo. Ovvero riesco a trovare la base dell'induzione sulla quale la proprietà è definita direttamente. L'induzione strutturale è usata per definire/dimostrare proprietà sui linguaggi di programmazione. L'idea centrale nell'induzione è quella di avere una base su cui definire/dimostrare direttamente (esplicitamente) la proprietà, e poi avere regole di costruzione/definizione che, assumendo definita/vera la proprietà per gli elementi usati, si definisce/dimostra la proprietà per l'elemento composto. Quindi si opera induttivamente sulle regole di costruzione dell'insieme.

Induzione matematica

Dato un insieme A ed una relazione binaria $< \subseteq A \times A$ ben fondata (senza catene discendenti infinite).

Se $A = \text{Nat}$ si ha *induzione matematica*

Base induttiva

Dato un insieme ben fondato $A (<)$.

Sia $\text{Base}_A = \{a \in A \mid a \text{ minimale}\}$

$\forall a \in A. \pi(a)$ sse

$\forall a \in \text{Base}_A. \pi(a) \wedge \forall a \in A \setminus \text{Base}_A. ([\forall b < a. \pi(b)] \Rightarrow \pi(a))$

ESEMPIO: INDUZIONE MATEMATICA

Dimostrare che $\sum_{i=1}^m i = \frac{m(m+1)}{2} \quad m \geq 1$

BASE: $m=1 \quad \sum_{i=1}^1 i = 1 = \frac{1(1+1)}{2} \quad \checkmark$

PASSO INDUTTIVO:

Supponiamo vero per m

(ipotesi induttiva $\sum_{i=1}^m i = \frac{m(m+1)}{2}$)

Dimostriamo per $m+1$: $\sum_{i=1}^{m+1} i = \frac{(m+1)(m+2)}{2}$

$$\sum_{i=1}^{m+1} i = \sum_{i=1}^m i + (m+1) = \frac{m(m+1)}{2} + m+1 =$$

per definizione di Σ per ipotesi induttiva

$$= \frac{m(m+1) + 2m+2}{2} = \frac{m^2 + m + 2m + 2}{2} =$$

$$= \frac{(m+1)m + (m+1)2}{2} = \frac{(m+1)(m+2)}{2} \quad \checkmark$$

INDUZIONE STRUTTURALE

Mentre l'induzione matematica, vista finora, permette di dimostrare proprietà di insiemi numerabili, dove possiamo identificare ogni elemento con un numero naturale su cui facciamo induzione, l'induzione strutturale permette più in generale di dimostrare proprietà di oggetti (ad es. insiemi) definiti in modo induttivo/ricorsivo.

Questo significa che su qualunque oggetto definito in modo induttivo possiamo fare induzione strutturale, ovvero dimostrare proprietà sulla struttura dell'oggetto, determinata dalla costruzione induttiva.

Supponiamo di definire un insieme X in modo induttivo: **(Base)** $0 \in X$; **(Passo induttivo)** Se $n \in X$ allora $n+3 \in X$. Questa definizione ci dice quali elementi stanno sicuramente in X (0) e come si ottengono elementi di X a partire da altri elementi di X . Grazie a questa definizione costruttiva, possiamo dimostrare proprietà degli elementi di X , come $X \subseteq \{3n \mid n \in \text{Nat}\}$.

La definizione costruttiva può essere usata anche per definire induttivamente funzioni sugli elementi dell'insieme. Vediamo un altro esempio.

Vogliamo definire l'insieme degli alberi binari BT con nodi n , usando un costruttore br :

$n \in \text{BT}$; Se $T_1, T_2 \in \text{BT}$ allora $br(T_1, T_2) \in \text{BT}$

dove br aggiunge un nodo come radice dei due sottoalberi T_1 e T_2 . Allora su questo insieme possiamo definire delle funzioni, e dimostrare proprietà

Definizioni: $foglie: \text{BT} \rightarrow \text{Nat}$ che conta le foglie di un albero: $foglie(n)=1$, $foglie(br(T_1, T_2))=foglie(T_1)+foglie(T_2)$; oppure la funzione che calcola il numero di diramazioni $branch: \text{BT} \rightarrow \text{Nat}$: $branch(n)=0$, $branch(br(T_1, T_2)) = branch(T_1) + branch(T_2) + 1$.

Dimostriamo: $\forall T \in \text{BT}. foglie(T) = branch(T) + 1$.

Dim: (Base) $foglie(n)=1=0+1=branch(n)+1$; (Passo) Suppongo per hp induttiva che per ogni $T \in \text{BT}$ la proprietà vale, dimostro che vale anche per l'oggetto costruito induttivamente: $foglie(br(T_1, T_2))=foglie(T_1)+foglie(T_2) = (\text{hp ind}) branch(T_1)+1+branch(T_2)+1 = (branch(T_1)+branch(T_2)+1) + 1 = branch(T)+1$ per definizione di $branch$.

INDUZIONE STRUTTURALE E GRAMMATICHE CF

Le grammatiche CF sono a loro volta strumenti per definire insiemi (linguaggi CF) in modo induttivo/strutturale. Questo significa che dalle grammatiche possiamo generare definizioni induttive e quindi dimostrazioni per induzione strutturale di proprietà degli insiemi (linguaggi) generati.

Prendiamo la seguente grammatica

$P \rightarrow aa \mid bb \mid aPa \mid bPb$

Da questa grammatica può essere derivata una definizione induttiva del linguaggio generato $SP=L(P)$

$aa, bb \in SP$; Se $x \in SP$ allora $axa, bxb \in SP$

A questo punto possiamo usare questa costruzione per dimostrare per induzione strutturale che tutte le stringhe in SP sono palindromi, ovvero

$x \in SP$ implica $\exists y, z. x = yz \wedge z = \text{reverse}(y)$.

Dim: (Base) $aa, bb \in SP$ e presi $y=a, z=a$, abbiamo che $aa=yz$ e $a=z=\text{reverse}(y)=a$.

(Passo induttivo) Supponiamo che sugli elementi di SP la proprietà valga, dimostriamo per il passo costruttivo.

Sia $x \in SP$, allora $axa \in SP$, inoltre per ipotesi induttiva, $\exists y, z$ tali che $x=yz$ e $z=\text{reverse}(y)$. Quindi $axa=ayza$. Se definiamo $y'=ay$ e $z'=za$, allora abbiamo che $axa=y'z'$, inoltre $\text{reverse}(z')=\text{reverse}(za)=a \text{ reverse}(z)=ay=y'$. Quindi abbiamo la tesi.

Vediamo un altro esempio

$E \rightarrow n \mid E + E \mid E * E$

Possiamo derivare la definizione induttiva delle espressioni $\text{EXP}=L(E)$ come

$n \in \text{EXP}$; Se $e_1, e_2 \in \text{EXP}$ allora $e_1+e_2 \in \text{EXP}, e_1*e_2 \in \text{EXP}$

Definiamo $op(e)$: $op(n)=0$; $op(e_1+e_2)=op(e_1)+op(e_2)$; $op(e_1)*e_2)=op(e_1)+op(e_2)+1$ e $val(e)$: $val(n)=1$; $val(e_1+e_2)=val(e_1)+val(e_2)$; $val(e_1)*e_2)=val(e_1)+val(e_2)+1$.

Dimostriamo che $\forall e \in \text{EXP}. val(e)=op(e)+1$

Dim: (Base) $val(n)=1=0+1=op(n)+1$

(Passo induttivo) $e=e_1+e_2$ (analogo e_1*e_2), allora

$val(e_1+e_2)=val(e_1)+val(e_2)=(\text{hp ind}) op(e_1)+1+op(e_2)+1 = (op(e_1)+op(e_2)+1)+1=op(e_1+e_2)+1$

UN SIGNIFICATO TANTE RAPPRESENTAZIONI

Abbiamo già parlato del fatto che ogni programma è una funzione matematica tra domini specifici. In particolare, un programma è solo una notazione per rappresentare funzioni. Infatti un algoritmo è una entità astratta concretizzata in un programma, ma una volta che abbiamo il programma non ci interessa se il programma implementa esattamente l'algoritmo, ma ci interessa che il programma implementi precisamente la funzione descritta dall'algoritmo. Il fatto che la funzione venga descritta mediante un algoritmo significa solamente che abbiamo in mente un modello computazionale che ci permette di suddividere la funzione in passi computazionali primitivi del modello computazionale. Quindi l'algoritmo è solo una dimensione intermedia che riempie il gap tra la funzione e il programma. In questo senso una strada è sicuramente quella di descrivere il significato di un programma caratterizzando la sua funzionalità. Questo è sicuramente quello che interessa al progettista che deve progettare i

costrutti del linguaggio per permettere l'implementazione di certe funzionalità e perché la composizione permetta di implementare certe funzionalità. D'altra parte l'implementatore ha sempre bisogno di descrivere funzionalità ma le deve descrivere come trasformazione di stato della macchina, fissati gli stati di input quali sono gli stati di output ottenuti dopo l'esecuzione del costruito? Dal punto di vista dello sviluppatore invece non interessa l'esatta trasformazione di stato, ma interessa come l'uso e la combinazione dei costrutti permetta di preservare invarianti o garantire proprietà desiderate. funzione.

Comportamento I/O? $\Rightarrow$ *Interessa l'implementatore*

Funzione descritta dall'algoritmo? $\Rightarrow$ *Interessa il progettista*

Proprietà - Invarianti? $\Rightarrow$ *Interessa lo sviluppatore*

Tutte queste visioni sono necessariamente collegate, nel senso che ogni funzione è una relazione e quindi una collezione di proprietà e viceversa ad ogni collezione di proprietà può essere associata una relazione o una funzione. Di fatto tutte queste rappresentazioni, tutti i punti di vista sono equivalenti, guardano solo al problema di rappresentare il significato da punti di vista diversi. I vari punti di vista hanno dato origine a diversi tipi di semantica che modellano i significati mediante strumenti matematici diversi (funzioni, transizioni di stato, proprietà) A seconda di quali strumenti matematici usiamo per descrivere i significati otteniamo tipi di semantiche diverse rendendo più agevoli tipi diversi di ragionamento. Le tipologie di strumenti, e quindi di semantiche, possono essere raccolte in tre macro classi:

Semantica denotazionale $\Rightarrow$ *Descrive funzionalità*

- **Denotazionale:** studia gli effetti dell'esecuzione, cerca proprietà del programma studiando proprietà della funzione calcolata

Semantica assiomatica $\Rightarrow$ *Descrive proprietà*

- **Assiomatica:** serve per fare deduzioni logiche a partire da assiomi dati, su parti del programma (dimostrazioni di correttezza)

Semantica operativa $\Rightarrow$ *Descrive trasformazioni di stato*

- **Operazionale:** si preoccupa di COME i risultati finali vengono prodotti (permette l'implementazione di un interprete)

Comunque tutte queste semantiche sono equivalenti, tale equivalenza è nota come full abstraction (problema molto sentito tra gli anni '60 e '70). La necessità di introdurre questi strumenti formali per descrivere i significati nasce dal fatto che i manuali di riferimenti spesso non erano sufficientemente dettagliati/formali per descrivere la funzionalità di certe caratteristiche dei linguaggi di programmazione e quindi spesso gli sviluppatori dovevano prendere decisioni autonome e indipendenti. Questo portava a spostare molti aspetti semantici nella pragmatica, ovvero tra le best practices, costringendo in fase di mantenimento del codice, a modificare o riscrivere programmi per spostarli da una macchina ad un'altra.

Esempio: Linguaggio L contiene solo assegnamento $:=$. Ogni forma di semantica descrive l'effetto/significato dell'esecuzione del programma sulla memoria.

```
P:
  z := 2;
  y := z;
  y := y+1;
  z := y;
```

SEMANTICA DENOTAZIONALE

Semantica denotazionale: Modello matematico dei programmi basato sulla ricorsione. È la semantica più “astratta” con cui descrivere i programmi.

Il processo di costruzione della semantica denotazionale per un linguaggio consiste nel definire un oggetto matematico per ogni entità del linguaggio e nel definire poi una funzione che mappa istanze delle entità del linguaggio in istanze dei corrispondenti oggetti matematici. Solitamente, il significato dei costrutti del linguaggio è definito solo in funzione del valore delle variabili del programma, ovvero in funzione della rappresentazione dello stato della macchina.

Formalmente, il modello matematico usato per descrivere la semantica denotazionale è quello delle funzioni matematiche (ricorsive), ovvero un programma corrisponde ad una funzione tra stati della macchina

$$\mathcal{E}: \text{Prog} \rightarrow ((\text{Var} \rightarrow \text{Val}) \rightarrow (\text{Var} \rightarrow \text{Val}))$$

e l'equivalenza di programmi si dimostra mediante equivalenza tra funzioni.

Quindi la semantica denotazionale è un oggetto puramente matematico che descrive gli effetti semplicemente manipolando oggetti matematici (si astrae da ogni concetto di esecuzione). In particolare, guarda agli effetti dell'esecuzione del programma, descrivendo l'effetto dell'esecuzione di una sequenza di comandi separati da ; attraverso la composizione degli effetti dei singoli comandi da sinistra verso destra. L'effetto di ogni comando è la funzione che dato uno stato produce un nuovo stato.

Questa semantica può essere usata per dimostrare la correttezza dei programmi, fornisce un modo formale per ragionare sui programmi, può aiutare della progettazione di linguaggi, viene usata anche in sistemi di generazione di compilatori. Purtroppo però, a causa della sua complessità ha poca utilità per gli utilizzatori dei linguaggi

Bisogna definire la funzione di **valutazione** $\mathcal{E}[P]\sigma$, che valuta il programma P sulla memoria σ , restituendo σ' , la memoria σ modificata da P

P:

```

z := 2;
y := z;
y := y+1;
z := y;
        
```

$$\begin{aligned}
 \mathcal{E}[P][z=\perp, y=\perp] = & \\
 (\mathcal{E}[z:=y] \circ \mathcal{E}[y:=y+1] \circ \mathcal{E}[y:=z] \circ \mathcal{E}[z:=2]) [z=\perp, y=\perp] = & \\
 (\mathcal{E}[z:=y] (\mathcal{E}[y:=y+1] (\mathcal{E}[y:=z] (\mathcal{E}[z:=2] [z=\perp, y=\perp]))) & \\
 \underbrace{\hspace{15em}}_{[z=2, y=\perp]} & \\
 \underbrace{\hspace{10em}}_{[z=2, y=2]} & \\
 \underbrace{\hspace{10em}}_{[z=2, y=3]} & \\
 \underbrace{\hspace{10em}}_{[z=3, y=3]} &
 \end{aligned}$$

SEMANTICA ASSIOMATICA

Semantica assiomatica: Modello matematico dei programmi basato sulla logica formale (calcolo dei predicati).

La semantica assiomatica nasce con l'obiettivo di fare verifica formale di programmi. Essa consiste di assiomi e regole di inferenza, fornite per ogni tipo di costrutto del linguaggio. Le espressioni logiche della semantica sono chiamate asserzioni:

- L'asserzione prima di un comando (precondizione) dichiara le relazioni e i vincoli validi prima dell'esecuzione del comando
- L'asserzione che segue il comando descrive cosa vale dopo l'esecuzione, ovvero è una postcondizione
- Una *weakest precondition* è la precondizione meno restrittiva che garantisce la postcondizione.

Attraverso queste asserzioni, la semantica assiomatica permette di dimostrare proprietà parziali di correttezza: quando lo stato iniziale rispetta la precondizione e il

programma termina, allora lo stato finale soddisfa la postcondizione.

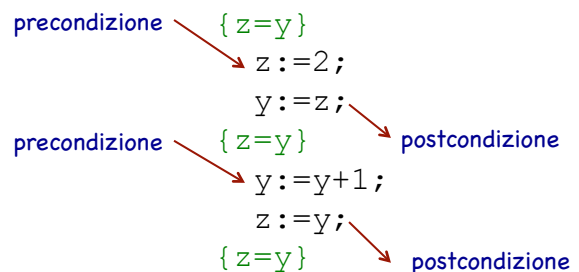
È chiaro che questo tipo di semantica considera solo alcuni aspetti dell'esecuzione del programma, ovvero quelli descritti dalle pre e post condizioni considerate. Questo significa che, date delle condizioni, esistono infiniti programmi che soddisfano tali condizioni, ma che hanno potenzialmente comportamenti diversi. D'altra parte, un programma può soddisfare diverse pre e post condizioni. Come ogni sistema logico, la semantica assiomatica deve godere di due importanti proprietà: *correttezza* (soundness), ovvero ogni proprietà derivabile nel sistema vale per il programma; *completezza*, ovvero ogni proprietà che vale per il programma è derivabile nel sistema di regole.

Notazione: $\{P\} \text{ statement } \{Q\}$. Questa semantica si calcola mediante un sistema di prova: abbiamo una tripla da verificare e cerchiamo di dimostrarla applicando le regole.

$$\frac{\{P\} S \{Q\}, P' \Rightarrow P, Q \Rightarrow Q'}{\{P'\} S \{Q'\}}$$
$$\frac{\{P1\} S1 \{P2\}, \{P2\} S2 \{P3\}}{\{P1\} S1; S2 \{P3\}}$$
$$\frac{\{B \text{ and } P\} S1 \{Q\}, \{\neg B \text{ and } P\} S2 \{Q\}}{\{P\} \text{ if } B \text{ then } S1 \text{ else } S2 \{Q\}}$$
$$\frac{(I \text{ and } B) S \{I\}}{\{I\} \text{ while } B \text{ do } S \{I \text{ and } (\neg B)\}}$$

P:

```
z := 2;  
y := z;  
y := y+1;  
z := y;
```



SEMANTICA OPERAZIONALE

Semantica operativa: Modello matematico dei programmi basato sui sistemi di transizione.

La semantica operativa descrive il significato del programma eseguendo i suoi comandi su una macchina, simulata o reale. La trasformazione di stato della macchina (memoria, registri, ecc) definisce il significato del comando. La semantica operativa di un linguaggio ad alto livello è necessaria per definire una macchina astratta che, induttivamente sulla struttura del programma a cui dare significato, va ad eseguire le sue componenti. La semantica operativa descrive il significato dei programmi come trasformazioni di stato e lo può fare a vari livelli di dettaglio/astrazione che comunque devono essere indipendenti dalla macchina/architettura. Lo stato è la rappresentazione astratta della macchina e quindi la sua formalizzazione è ciò che decide il livello di astrazione a cui guardare il comportamento run-time dei costrutti.

Dal punto di vista del modello, la semantica operativa si preoccupa di come i risultati finali vengono calcolati, in

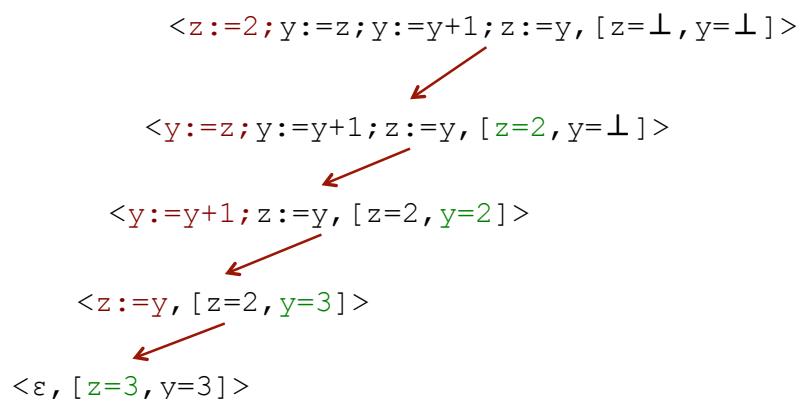
particolare, il modello matematico usato è quello dei sistemi di transizione.

Si consideri la funzione **memoria** $\sigma: \text{Var} \rightarrow \text{Val}$ che associa valori alle variabili,. Lo stato è una coppia che consiste nel programma ancora da eseguire e nello stato in cui si deve eseguire il programma: Stato = $\langle P, \sigma \rangle$. Ad ogni passo si esegue un'operazione e si cambia configurazione/stato: applicando una relazione tra configurazioni $\rightarrow \subseteq \langle P, \sigma \rangle \times \langle P, \sigma \rangle$ (relazione di transizione). La chiusura transitiva descrive l'esecuzione completa. Tale semantica opera eseguendo i comandi separati da ; sequenzialmente e nell'ordine in cui compaiono da sinistra a destra.

Anche se è una delle forme più concrete di semantica, essa esegue comunque l'astrazione di come il programma viene eseguito (architettura, registri, indirizzi), essa è quindi comunque indipendente dall'architettura.

Esempio iniziale:

```
P:  
  z := 2;  
  y := z;  
  y := y+1;  
  z := y;
```



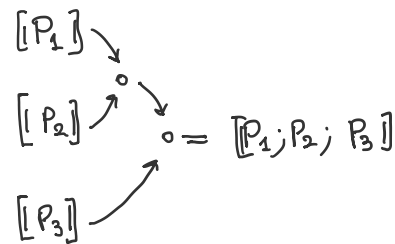
COMPOSIZIONALITÀ

Composizionalità: Il significato di ogni programma deve essere funzione del significato dei costituenti immediati.

La composizionalità è una proprietà della semantica necessaria per caratterizzare i comportamenti e significati di sistemi che possono avere infiniti elementi. Si noti che la semantica denotazionale e la semantica assiomatica rispettano banalmente tale principio. La composizionalità della semantica operativa è invece meno immediata (Plotkin).

La semantica è alla base di ogni analisi che si può fare su un programma. Per analizzare le proprietà di un programma devo capire cosa fa e quindi devo capire la semantica in qualche sua forma. La composizionalità in tal caso diventa essenziale per garantire la modularità all'analisi di programmi, il che garantisce di poter ragionare in maniera modulare anche su software di grandi

dimensioni o comunque sviluppato modularmente. Infatti, se la semantica su cui si basa una analisi è composizionale, allora è possibile analizzare il software separatamente nei suoi moduli, per poi ricomporre il risultato dell'analisi componendo i risultati ottenuti sui singoli moduli.



EQUIVALENZA

Equivalenza: Due programmi sono equivalenti quando hanno la stessa semantica (funzionalità, esecuzione, proprietà?)

Sappiamo che la stessa funzione può essere calcolata seguendo algoritmi diversi che possono manipolare anche dati diversi, e poi per ogni algoritmo esistono infiniti programmi che lo implementano, quindi è importante confrontare algoritmi e programmi. Questo significa che la semantica non serve solo a sapere come usare un linguaggio, ma serve anche a confrontare programmi. È chiaro che alla base di ogni confronto c'è la caratterizzazione della funzione calcolata, e quindi i programmi possono essere confrontati quando calcolano la stessa funzione. Questo vuol dire vedere il programma come una scatola nera e guardare esclusivamente alla relazione di input/output (I/O), se tale relazione è la stessa,

allora indipendentemente da come (algoritmo) la calcolano (dentro la scatola nera) la funzionalità è la stessa. Solo sapendo caratterizzare la funzionalità I/O possiamo quindi dimostrare equivalenze di programmi. A questo punto, solo quando sappiamo se due programmi calcolano la stessa funzione li possiamo confrontare rispetto all'efficienza, quando possibile (potrebbero non essere confrontabili rispetto alla stessa risorsa tempo/spazio).

L'equivalenza è necessaria in varie fasi di analisi:

Correttezza (per dimostrare che il programma scritto calcola esattamente la funzione attesa)

Equivalenza di programmi (dimostrare che due programmi calcolano la stessa funzione)

Efficienza (Dati due programmi che calcolano la stessa funzione, dimostrare quale lo fa in modo più efficiente)

SINTASSI: CATEGORIE SINTATTICHE

- Cosa sono?
- Come vengono caratterizzate?
- Cosa le distingue?

- Ritorniamo ora alla sintassi, ma da un punto di vista semantico, ovvero da un punto di vista indipendente dalla macchina, dall'implementazione. Quindi ci poniamo il problema di capire quali classi sintattiche deve avere un linguaggio per essere un linguaggio di programmazione.
- Per fare questo classifichiamo i costrutti (che dal punto di vista formale sono generati da non terminali della grammatica) in categorie sintattiche base, la cui interpretazione è chiaramente legata al significato atteso del costrutto stesso.
- Per descrivere tutte le categorie sintattiche che vedremo, è essenziale il concetto di funzione, che non deve essere troppo concreto (step by step, vicino alla macchina), ma neanche troppo astratto (I/O perdendo il legame con la computazione).
- Questa funzione deve descrivere l'effetto della categoria sintattica sullo stato della macchina, quindi per distinguere le categorie abbiamo bisogno per prima cosa di introdurre il concetto di stato, con tutti gli elementi che ci permettono di distinguere tra loro le categorie sintattiche.

STATO (AMBIENTE + MEMORIA)

È chiaro che finché non associamo ai nomi ciò che rappresentano non possiamo parlare di significato di programmi. Per descrivere adeguatamente queste associazioni abbiamo bisogno di due entità, ambiente e memoria. Capiremo piano piano questa distinzione, necessaria per gestire sia associazioni che non cambiano (ambiente), sia associazioni che cambiano (memoria) durante l'esecuzione del programma.

L'ambiente specifica quali nomi sono usati e per quali oggetti, solitamente possono essere legati ad un tipo, ad un valore, ad una locazione. Quindi l'ambiente specifica i nomi e quali *binding* sono associati a questi nomi, che possono rappresentare variabili, costanti, procedure, ecc. È essenziale sottolineare che il *nome* quindi è un'entità separata dall'oggetto che denota, infatti lo stesso nome/identificatore può essere usato in contesti (scope) diversi, ovvero può avere binding diversi in diversi momenti del programma, non solo può identificare valori diversi, ma

Ambiente (environment)
Insieme di legami (bindings) tra identificatori e denotazioni.

Memoria (store)
Insieme di effetti sugli identificatori (causati da assegnamenti)

anche oggetti completamente diversi. Questo significa che i binding possono non valere per tutta la durata del programma ma possono avere un "tempo di vita".

La memoria è una mappa che solitamente rappresenta la storia, evoluzione, delle variazioni dei valori associati agli identificatori. Ovviamente può non tenere traccia dell'intera storia ma può descrivere solo l'effetto dell'ultimo aggiornamento eseguito. Quindi fornisce un binding tra locazione e valore (è l'ambiente che mi dice a quale locazione in ogni istante un nome si lega). Si tenga presente comunque che questo concetto di memoria è fortemente legato all'approccio imperativo, infatti i linguaggi funzionali puri non ne hanno bisogno, non dovendo gestire variabili che cambiano durante l'esecuzione del programma.

Adesso abbiamo gli ingredienti per parlare delle categorie sintattiche dei linguaggi imperativi.

CATEGORIE SINTATTICHE

Categorie sintattiche: classificazione dei costrutti in funzione del loro significato atteso, ovvero della classe di effetti che la loro esecuzione causa.

Le categorie sintattiche quindi sono classi che rappresentano un diverso tipo di significato, di effetto, dell'esecuzione di un programma. In generale possiamo identificare tre fondamentali effetti derivanti dall'esecuzione di un programma:

- Generazione di *valori*
- Creazione di *legami*
- Trasformazioni di *stato*

Ogni diverso significato si associa a un diverso costrutto, quindi per descrivere le categorie sintattiche dobbiamo descrivere il loro significato e per fare questo dobbiamo usare gli elementi che ci permettono di descrivere i significati in funzione dello stato.

Le categorie sintattiche necessarie per parlare di linguaggi di programmazione si distinguono in funzione di cosa *denotano/producono/ottengono* rispetto ad uno stato di computazione. Partendo da questo otteniamo esattamente tre categorie: **Espressioni, Comandi e Dichiarazioni**. Queste classi di elementi sono disgiunte, nel senso che una dichiarazione tipicamente deve essere ben distinta da un comando e da una espressione e viceversa (anche se esistono linguaggi con qualche sovrapposizione), questo comunque non significa che alcune espressioni non possano avere come side-effect l'effetto di un comando, un comando di una dichiarazione, ecc.

Dal punto di vista formale, ovvero dal punto di vista della definizione del linguaggio, le categorie sono i simboli non terminali della grammatica classificati in funzione di cosa modificano dello stato e come lo modificano.

ESPRESSIONI

Denotano VALORI

I programmi fondamentalmente manipolano dati, ovvero valori, ma per manipolare valori dobbiamo necessariamente rappresentarli e quindi in un linguaggio di programmazione abbiamo bisogno per prima cosa di una categoria sintattica che permetta di rappresentare/denotare valori. Per questo introduciamo le espressioni. Quindi, il significato di una espressione è essenzialmente un valore, potenzialmente con dei vincoli (ad esempio il tipo), ma comunque un valore.

Le **espressioni** devono essere **valutate** per restituire un valore, e va osservato che il concetto di valore è diverso da quello di locazione di memoria: i valori sono parte dello specifico linguaggio di programmazione (linguaggi di programmazione diversi possono manipolare valori diversi), la locazione è parte della struttura dell'architettura, anche se modella il modo con cui il linguaggio di programmazione si relaziona alla macchina sottostante.

Due espressioni possono essere *DIVERSE* ma essere valutate nello stesso *VALORE* (in *tutti gli stati*). Ad esempio, $\text{not}(a \text{ and } b)$ è logicamente (semanticamente) equivalente a $(\text{not } a) \text{ or } (\text{not } b)$ (De Morgan) ma sono sintatticamente diverse. Quindi usiamo oggetti sintattici ma ragioniamo sempre sul loro significato, che è ciò che a noi interessa, la sintassi è solo lo strumento per ragionare sul significato, come le lingue sono solo lo strumento per comunicare gli stessi significati in paesi diversi.

Cosa significa quindi per due espressioni essere *equivalenti*? Per essere equivalenti non basta denotare lo stesso valore in stati specifici, ma è necessario denotare lo stesso valore in tutti gli stati possibili.

Equivalenza

Due espressioni sono equivalenti se vengono valutate nello stesso valore in tutti gli stati di computazione (anche eventuali side-effects devono essere gli stessi).

DICHIARAZIONI

Denotano richieste di MODIFICA/ CREAZIONE di LEGAMI

Ora che sappiamo come rappresentare valori, vogliamo poter memorizzare tali valori per manipolarli successivamente. È chiaro che per memorizzarli e riferirli successivamente, dobbiamo associare dei nomi/identificatori ai valori, ovvero dobbiamo creare/usare/modificare **legami** tra nomi e valori (in generale oggetti denotabili). L'insieme dei legami da utilizzare costituiscono l'ambiente (ritorneremo su questi concetti più avanti).

Le dichiarazioni sono la categoria sintattica che permette la creazione o la modifica dei legami associati agli identificatori, ovvero gli ambienti. Va notato che le modifiche agli ambienti sono trasformazioni **reversibili**, in quanto le trasformazioni valgono esclusivamente all'interno del raggio di azione (*scope*) attuale dell'identificatore. In altre parole, i linguaggi di programmazione permettono di delimitare la validità di un ambiente, quindi tutte le modifiche all'ambiente sono reversibili nel senso che quando termina la sua validità

tutte le modifiche vengono annullate automaticamente, scompaiono con la terminazione della validità dell'ambiente. La creazione di ambiente quindi crea sempre nomi nuovi e locali, anche se sono nomi uguali ad altri usati nel programma.

In questo caso, in cosa consiste l'equivalenza? Sicuramente due dichiarazioni equivalenti devono generare le stesse richieste, ma, più di altre categorie sintattiche, le dichiarazioni possono avere side-effect, in tal caso la dichiarazione potrebbe inizializzare una variabile e quindi modificare la memoria, per questo l'equivalenza deve necessariamente richiedere che due dichiarazioni equivalenti generino le stesse identiche modifiche a tutto lo stato di computazione.

Equivalenza

Due dichiarazioni sono equivalenti se producono lo stesso ambiente (e la stessa memoria in caso di side-effects) in tutti gli stati di computazione

COMANDI

Denotano richieste di MODIFICA della MEMORIA

Ora che sappiamo come rappresentare i valori e sappiamo come associare loro dei nomi che mi permettono di memorizzarli e riferirli in durante la computazione, dobbiamo avere uno strumento per manipolarli: i *comandi*.

I comandi sono richieste di modifica dello stato di computazione, ed in particolare della memoria. Queste trasformazioni sono **irreversibili**, ovvero definitive nell'esecuzione del programma, per annullarle, se possibile devo eseguire altri comandi, altre trasformazioni che possono riportarmi alla stessa memoria iniziale.

I **comandi** rappresentano quindi funzioni di trasformazione e devono essere **eseguiti** per poter attuare la corrispondente trasformazione (*irreversibile*) della memoria.

Anche i comandi possono essere confrontati, ed in particolare ci chiediamo quindi quando due comandi sono equivalenti. Intuitivamente, poiché un comando esegue

una trasformazione dello stato, due comandi sono equivalenti se rappresentano la stessa funzione di trasformazione, quindi se per ogni stato in input restituiscono sempre lo stesso stato in output.

Si noti che anche in questo caso l'equivalenza è una proprietà universale sui possibili input.

Equivalenza

Due comandi sono equivalenti se per ogni stato (memoria) in input producono lo stesso stato (memoria) in output

RIASSUMENDO

Le categorie sintattiche necessarie per parlare di linguaggi di programmazione sono:

- **Espressioni** ⇒ *Vengono valutate in valori.*

Nella loro forma pura non modificano lo stato di computazione e gli ambienti

- **Dichiarazioni** ⇒ *La loro elaborazione crea/modifica ambienti/legami (modifiche reversibili) tra nomi e valori.*

Nella loro forma pura non producono valori e non modificano la memoria

- **Comandi** ⇒ *Eseguono modifiche irreversibili della memoria.*

Nella loro forma pura non producono valori e non modificano ambienti.

Comunque, tutte le categorie sintattiche possono generare effetti collaterali, una dichiarazione che inizializza trasforma anche la memoria, la chiamata ad una funzione è un comando che produce un valore, ma se usato in un assegnamento è una espressione che esegue potenzialmente (nella procedura chiamata) trasformazioni dello stato, un assegnamento (in C) può produrre un valore, quello assegnato alla variabile, nei linguaggi non tipati l'assegnamento è un comando che genera anche binding di tipo, ecc...

Queste tre categorie sono presenti in tutti i linguaggi di base imperativa, ovvero che fondano la computazione sull'assegnamento, ma va notato che in altri paradigmi non è sempre così. Ad esempio nei linguaggi funzionali abbiamo solo espressioni e dichiarazioni, non abbiamo il concetto di stato, e quindi di trasformazione di stato, per cui i comandi non sono necessari. La computazione in tal caso si basa sul concetto di applicazione di funzioni.

SEMANTICA OPERAZIONALE

- Cosa va descritto?
- Con quali strumenti?

- Abbiamo bisogno di un framework per descrivere la semantica operativa, partendo da quello che abbiamo, ovvero regole di produzione della grammatica, Parse trees e abstract syntax trees (dove non c'è ambiguità)... ogni costrutto (in particolare le espressioni) viene dato assumendo il suo abstract syntax tree che non ha ambiguità (rivedremo questa cosa dopo nella sintassi del linguaggio).
- Il significato operativo consiste nel tradurre i passi computazionali in una forma indipendente dall'architettura/macchina ma che modelli l'evoluzione del suo stato, descritto come ambiente e memoria.
- Vedremo che le modifiche alla memoria sono modifiche *irreversibili* dello stato, ovvero sono necessari ulteriori passi computazionali, ovvero uno sforzo computazionale, per annullare gli effetti di una modifica. Esistono anche modifiche reversibili dello stato, ovvero per le quali non serve uno sforzo computazionale (extra) per annullare gli effetti della modifica, queste sono le modifiche sull'ambiente (ottenute elaborando le dichiarazioni)

SISTEMI DI TRANSIZIONE

Visto che la semantica deve essere specificata in funzione della sintassi allora il modo migliore per descrivere la semantica è attraverso la manipolazione di simboli.

Possiamo descrivere la manipolazione dei simboli che effettua il compilatore, quindi descriviamo esattamente la manipolazione del linguaggio sorgente effettuata dal compilatore per arrivare poi a generare il codice eseguibile, ma questo è anche troppo specifico se il nostro obiettivo è solo sapere cosa fa un costrutto, non ci interessa esattamente come il compilatore lo riconosce e lo traduce in operazioni primitive della macchina sottostante. Possiamo descrivere la collezione di algoritmi corrispondenti ad ogni costrutto, ma anche questo è troppo specifico, anche se in modo diverso. Ci interessa cosa fa un costrutto non come lo fa, quindi siamo troppo vicino all'implementazione del costrutto. Possiamo descrivere in linguaggio naturale il comportamento dei costrutti, ma questo comporta ovvi problemi di

interpretazioni potenzialmente conflittuali, dovute al linguaggio naturale usato.

In generale i primi due metodi spiegano COME agiscono e non COSA fanno i costrutti, questo non va bene perché il significato di un oggetto *non* è il metodo implementato per eseguirlo/interpretarlo. Inoltre non sono concisi e neanche abbastanza astratti (vengono dati dettagli irrilevanti, senza necessariamente specificare ciò che l'utente ha necessità di sapere). Sono spesso legati all'architettura su cui il linguaggio viene eseguito. Il terzo metodo non è preciso ed è ambiguo a causa del linguaggio naturale, è verboso, non è accurato (formale) nella descrizione del significato ma specifica il COSA e non il COME. Quindi abbiamo bisogno di uno strumento formale che simula le trasformazioni che vengono eseguite ad un livello sufficientemente astratto da ignorare i dettagli del COME descrivendo quindi solo il COSA viene calcolato,.

Lo strumento formale utilizzato è quello dei sistemi di transizione. Le loro principali caratteristiche sono:

- Sono matematicamente precisi
- Sono molto concisi
- Sono un metodo di specifica generale e che permette astrazione
- Sono espressi mediante una collezione di regole date in funzione della sintassi (induttivamente): **Specificano cosa viene calcolato per induzione sulla struttura sintattica del linguaggio.**

I sistemi di transizione sono sufficientemente astratti per specificare senza ambiguità e senza dipendenza dalla macchina, cosa fa un linguaggio. Sfrutta la definizione induttiva del linguaggio, permettendo di definire il significato mediante induzione sull'abstract syntax tree, quindi si usa l'induzione strutturale matematica per ragionare sui programmi. È uno strumento molto generale, infatti qualunque sistema di computazione può essere espresso mediante un sistema di transizione (sistemi HW, automi, ecc), inoltre, come abbiamo detto, astrae dettagli a basso livello, permettendo di modellare l'equivalenza

denotazionale e di modulare (adattare al contesto) la precisione dell'osservazione.

Sistema di transizione:

Un sistema di transizione è una struttura $(\Gamma, \rightarrow)$, dove Γ è un insieme di elementi γ chiamati configurazioni e la relazione binaria $\rightarrow \subseteq \Gamma \times \Gamma$ è chiamata relazione di transizione. Se $\Gamma_T \subseteq \Gamma$ è un insieme di configurazioni terminali, il sistema è detto terminale.

Le configurazioni dipendono dal linguaggio, in particolare dal tipo di linguaggio. Quando due configurazioni sono nella relazione di transizione diciamo che la prima si muove o si può muovere nella seconda. Il sistema di transizione genera un multigrafo diretto infinito, dove le configurazioni sono nodi, mentre le transizioni sono archi diretti. L'esecuzione di un programma, esattamente come la sintassi, può essere rappresentato mediante un grafo, i cui nodi sono le configurazioni.

Notazione:

$$\gamma \rightarrow \gamma' \text{ per } \langle \gamma, \gamma' \rangle \in \rightarrow$$

$$\gamma_0 \rightarrow^* \gamma_n \Leftrightarrow \exists \gamma_1 \dots \gamma_n. \gamma_i \rightarrow \gamma_{i+1}, i \in [0, n]$$

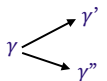
Una derivazione è una sequenza di transizioni. Il sistema di transizione definisce come le derivazioni avvengono, ovvero come si possono raggiungere le configurazioni finali (stringhe di soli simboli terminali). Quindi il sistema di transizione è il framework di base, il come viene definito dipende da quello che si vuole descrivere. L'evoluzione delle configurazioni specifica cosa accade e non come questo accade, anche se il livello di dettaglio può includere aspetti anche di metodo. In generale il modello basato su sistema di transizione funziona bene, perché per definire la semantica in modo induttivo sulla struttura sintattica, non ci interessa specificare esecuzioni complete ma specificare regole locali che, combinate tra loro, descrivono esecuzioni complete. Inoltre, le regole dicono cosa si può inferire e non cosa NON si può inferire, quello che non si può fare/descrivere si ottiene implicitamente dall'impossibilità di derivarlo a partire dalle regole, dall'impossibilità di dimostrare il fatto mediante l'uso delle regole definite. Inoltre le regole permettono di descrivere ciò che può avvenire e non ciò che deve avvenire, ovvero non si precludono possibilità diverse previste dalle regole.

Come esempio vediamo come si può generare un sistema di transizione a partire da una grammatica CF.

$$G = \langle V, T, P, S \rangle, \Gamma = (V \cup T)^*$$

$$A \rightarrow \alpha \quad \Rightarrow \quad \frac{A \rightarrow \alpha}{\beta A \gamma \rightarrow \beta \alpha \gamma}$$

Si noti che, la relazione di transizione in un sistema di transizione è una *relazione* non una funzione, quindi comportamenti non deterministici sono possibili. Il non determinismo deriva dal non poter dare troppi dettagli che permettano di individuare un'unica soluzione. In questo contesto il non-determinismo è intrinseco sia nel problema che nelle possibili soluzioni. Quindi la semantica è in generale non deterministica, anche quando modella sistemi deterministici, in quanto il modello non fornisce (o meglio non vuole fornire) sufficienti dettagli per determinare la descrizione della computazione. Al contrario l'implementazione, che ha tutti i dettagli, deve essere deterministica.



PLO - UN SEMPLICE LINGUAGGIO REALE

- Ma come è fatto un linguaggio reale?
- Come sono fatte le categorie sintattiche nella loro forma più semplice

- Linguaggio Pascal-like
- Singolo tipo di dato INTEGER (Booleani rappresentati come interi)
- Strutture di controllo standard
- Astrazione del controllo (Procedure senza parametri)

```

<program>  P → B.
<block>    B → D S
<declar>   D → [const I=N{,I=N};]
            | [var I{,I};]
            | [procedure I;B;]
<stmts>    S → ε | I := E
            | call I
            | if C then S
            | while C do S
            | begin S{;S} end
<Cond>     C → odd E | E > E
            | E >= E | E = E
            | E # E | E < E
            | E <= E
<Expr>     E → I | N | E bop E
            | ( E )
  
```

SINTASSI DI PLO

- I programmi P sono blocchi.
- I blocchi B sono una dichiarazione D seguita da un comando S. Si noti che D ed S sono definiti in modo libero dal contesto, ovvero quello che possiamo generare da S non deve avere necessariamente nessun vincolo con quello che generiamo da D. In altre parole questa grammatica non ci obbliga in nessun modo ad usare identificatori precedentemente definiti. Quindi dalla descrizione sintattica rimangono fuori gli aspetti context sensitive (vincoli sintattici) che la grammatica CF non è in grado di rappresentare. Servirà la definizione di una "semantica statica" che avrà il ruolo di verificare questi vincoli prima di compilare e/o eseguire.
- Le dichiarazioni D sono dichiarazioni di valori costanti con nome (const), dichiarazioni di variabili (var) e dichiarazioni di procedure (ovvero di un blocco riferibile con un nome).

- Le condizioni C sono espressioni dal valore booleano: Odd è un predicato unario che verifica se l'espressione è 0, poi ci sono vari operatori di confronto di espressioni E.
- Le espressioni E sono identificatori, valori naturali, operazioni tra espressioni e espressioni tra parentesi.
- Qui sintetizziamo con op la metavariable dell'insieme {+,-,*,/}. La stessa cosa si potrebbe fare per gli operatori di confronto, mettendo in C la produzione E cop E con cop metavariable per gli elementi in { >, >=, =, #, <=, < }.

Per la grammatica delle espressioni dobbiamo fare attenzione all'ambiguità: questa è ambigua non usando le parentesi. Se invece mettiamo tutte le parentesi, allora non abbiamo più l'ambiguità ma diventa un linguaggio pesante da usare, pronò ad errori (tante parentesi da scrivere) e difficile da leggere. Dobbiamo quindi cercare un compromesso che permetta di tener conto le precedenze matematiche in assenza di parentesi, senza obbligare ad usare le parentesi e comunque eliminando l'ambiguità.

- I nomi sono gli identificatori I, considerati qui parte del vocabolario; N sono i numeri naturali, considerati anch'essi parte del vocabolario.
- Non abbiamo bisogno del concetto di tipo in quanto abbiamo solo un tipo di dato.
- Si noti che le produzioni di D sono tutte opzionali, questo significa che un programma può essere un blocco senza dichiarazioni.
- I comandi sono: il comando vuoto, l'assegnamento di un valore (denotato da una espressione) ad un identificatore, la chiamata ad una procedura mediante il suo nome, il comando condizionale che esegue un comando al verificarsi di una condizione booleana C, il ciclo che ripete un comando finché una data condizione C è vera e la composizione sequenziale di comandi.

```
<Expr>      E → [+,-] T [A T]
<Add op>    A → + | -
<Terms>     T → F [M F]
              M → * | /
<Factor>    F → I | N | ( E )
<Numbers>   N → [+,-] d{d}
<Digit>     d → 0 | ... | 9
<Ident>     I → l{l,d}
<Letter>    l → A | ... | Z
```

Questa grammatica delle espressioni rispetta tutte le convenzioni e non è ambigua. Questa è la definizione che serve ai compilatori, mentre per parlare di semantica possiamo anche semplicemente usare la grammatica ambigua, più facile da leggere e comprendere.

Per il compilatore, la grammatica deve essere completa, quindi dobbiamo specificare tutti gli elementi. L'identificatore è una stringa che deve necessariamente iniziare con un carattere non numerico. Qui ci sono solo i caratteri maiuscoli, ma è molto facile estendere. È da notare che questo ultimo gruppo di grammatiche (N e I) non sono parte del parsing ma dell'analisi lessicale (scanning), essendo linguaggi regolari, per questo non sono tipicamente considerati parte della grammatica principale. Ricordiamo che lo scanning prende il programma e lo divide in parole/lessemi che descrivono entità del programma (parole, identificatori, costanti..) e poi lo si guarda come un singolo elemento. Questo è usato anche per raccogliere le informazioni sui tipi, la fase di scan quindi fa l'operazione di convertire una sequenza di caratteri in una sequenza di lessemi, ovvero di elementi corredati dal "tipo". Questo significa che, se ci mettiamo al livello della grammatica dei comandi, allora possiamo trattare semplicemente identificatori e numeri come insiemi noti $N = \text{INTEGERS}$, $I = \{\text{id, rate, ciao2, ...}\}$. Questo è un esempio di come la potenza delle CFG permetta di modificare la grammatica, ottenendo nuove grammatiche equivalenti ma più adatte magari al processo di elaborazione per il quale vengono definite (compilazione). Quindi le CFG permettono di descrivere i linguaggi ad un

profondo livello di dettaglio che non è necessario conoscere per utilizzare il linguaggio. Solitamente l'utente deve conoscere la sintassi astratta del linguaggio. Quindi i punti di vista dell'implementatore e dell'utente sono diversi: è necessaria una grammatica abbastanza grossolana, magari con ambiguità, per l'utente che è interessato solo all'abstract syntax tree, dove serve sapere se scrivendo in un modo o in un altro ci sono differenze ****semantiche****, ma si necessita una grammatica abbastanza fine per evitare problemi di ambiguità ad esempio nell'implementazione di un parser.